EXPERIMENT - 72

1. Problem Statement

Develop a C program to manage student attendance using a dynamically resizable matrix, with pointers and no fixed-size 2D array. Each row is one student and each column is one working day (1 = Present, 0 = Absent). The program must:

- Accept the initial students and working days, use **malloc()** for the initial matrix, and use **realloc()** when more students are added.
- Accept only 0 or 1 as attendance and display the attendance percentage of each student.
- Use separate functions for creating the matrix, recording attendance, adding students, displaying records, calculating percentages and releasing memory.
- Handle invalid input and allocation failure, and **free()** all memory before the program ends.

2. Algorithm

Main Program

Step 1: START
Step 2: Read students and days with read_number(), which asks again until the number is at least 1. Call create_matrix(); if it returns NULL, display Memory allocation failed and stop.
Step 3: Call record_attendance() for the initial students.
Step 4: Repeat: read extra (0 to stop). If extra > 0, call add_students(). Stop repeating when extra = 0 or adding fails.
Step 5: Call display_attendance() and calculate_percentages(), then free_matrix().
Step 6: STOP

Functions

create_matrix(students, days): malloc() students × days integers and return the pointer.
record_attendance(matrix, first, last, days): For each student from first to last − 1, read days values. If any is not 0 or 1, display Invalid input and read that student again. Store each value at *(matrix + s * days + d).
add_students(matrix, students, days, extra): realloc() the matrix to (students + extra) × days integers. If it returns NULL, display a failure message and keep the old matrix. Otherwise record the new students and add extra to students.
display_attendance(): Display the header and one row of values for each student.
calculate_percentages(): For each student, count the 1 values, find present × 100 / days and display it.
free_matrix(): free() the matrix and set the pointer to NULL.

3. Test Case Table

#	Input	Expected Output	Actual Output	Result
1	3 students, 3 days: 1 0 0 / 1 0 1 / 1 1 1. Add 3: 1 0 0 / 1 1 1 / 1 0 1 (the example in the question)	Student-1: 33.3%, Student-2: 66.7%, Student-3: 100%, Student-4: 33.3%, Student-5: 100%, Student-6: 66.7%	Student-1: 33.3%, Student-2: 66.7%, Student-3: 100%, Student-4: 33.3%, Student-5: 100%, Student-6: 66.7%	**PASS**
2	1 student, 3 days. First entry 1 2 1 (2 is invalid), then 1 1 0	Invalid input. Only 0 or 1 is allowed. Student-1: 66.7%	Invalid input. Only 0 or 1 is allowed. Student-1: 66.7%	**PASS**
3	Number of students 0 (invalid), then 2. 2 days: 1 1 / 0 1	Invalid input. Enter a number that is at least 1. Student-1: 100%, Student-2: 50%	Invalid input. Enter a number that is at least 1. Student-1: 100%, Student-2: 50%	**PASS**
4	1 student, 2 days: 1 0. Add 1 student: 1 1. Add 2 students: 0 0 / 1 0	Student-1: 50%, Student-2: 100%, Student-3: 0%, Student-4: 50%	Student-1: 50%, Student-2: 100%, Student-3: 0%, Student-4: 50%	**PASS**

4. Source Code

attendance72.c

```
/* Experiment 72: student attendance in a dynamically resizable matrix.
   Rows are students and columns are working days (1 = Present, 0 = Absent).
   The matrix is one malloc()ed block, so the element for student s on
   day d is *(matrix + s * days + d). */
#include <stdio.h>
#include <stdlib.h>

int read_number(const char *prompt, int min);
int *create_matrix(int students, int days);
int record_attendance(int *matrix, int first, int last, int days);
int add_students(int **matrix, int *students, int days, int extra);
void display_attendance(int *matrix, int students, int days);
void calculate_percentages(int *matrix, int students, int days);
void free_matrix(int **matrix);

int main(void)
{
    int students, days, extra;
    int *matrix;

    students = read_number("Enter the number of students: ", 1);
    days = read_number("Enter the number of working days: ", 1);
    if (students < 1 || days < 1) {
        printf("No valid input received.\n");
        return 1;
    }

    matrix = create_matrix(students, days);
    if (matrix == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }
    if (!record_attendance(matrix, 0, students, days)) {
        free_matrix(&matrix);
        return 1;
    }

    while ((extra = read_number("Enter the number of students to add (0 to stop): ", 0)) > 0)
        if (!add_students(&matrix, &students, days, extra))
            break;

    display_attendance(matrix, students, days);
    calculate_percentages(matrix, students, days);
    free_matrix(&matrix);
    return 0;
}

/* Reads a whole number that is at least min. Returns -1 at end of input. */
int read_number(const char *prompt, int min)
{
    int value, result, ch;

    while (1) {
        printf("%s", prompt);
        result = scanf("%d", &value);
        if (result == EOF)
            return -1;
        if (result == 1 && value >= min)
            return value;
        while ((ch = getchar()) != '\n' && ch != EOF)
            ;
        printf("Invalid input. Enter a number that is at least %d.\n", min);
    }
}

int *create_matrix(int students, int days)
{
    return (int *)malloc((size_t)students * days * sizeof(int));
}
```

```c
/* Records the rows first to last - 1, one student per line. A line with
   anything other than 0 or 1 is rejected and asked for again. */
int record_attendance(int *matrix, int first, int last, int days)
{
    int s, d, value, result, ok, ch;

    for (s = first; s < last; s++) {
        do {
            ok = 1;
            printf("Enter attendance of Student %d (1 = Present, 0 = Absent): ", s + 1);
            for (d = 0; d < days; d++) {
                result = scanf("%d", &value);
                if (result == EOF)
                    return 0;
                if (result != 1) {
                    ok = 0;
                    break;
                }
                if (value != 0 && value != 1)
                    ok = 0;
                *(matrix + s * days + d) = value;
            }
            if (!ok) {
                while ((ch = getchar()) != '\n' && ch != EOF)
                    ;
                printf("Invalid input. Only 0 or 1 is allowed.\n");
            }
        } while (!ok);
    }
    return 1;
}

/* Grows the matrix with realloc(). If that fails, the old matrix is kept. */
int add_students(int **matrix, int *students, int days, int extra)
{
    int *bigger = (int *)realloc(*matrix, (size_t)(*students + extra) * days * sizeof(int));

    if (bigger == NULL) {
        printf("Memory allocation failed. No students were added.\n");
        return 0;
    }
    *matrix = bigger;
    if (!record_attendance(*matrix, *students, *students + extra, days))
        return 0;
    *students += extra;
    return 1;
}

void display_attendance(int *matrix, int students, int days)
{
    int s, d;

    printf("Attendance records:\n");
    printf("%-10s", "Student");
    for (d = 0; d < days; d++)
        printf("Day %-3d", d + 1);
    printf("\n");
    for (s = 0; s < students; s++) {
        printf("Student %-2d", s + 1);
        for (d = 0; d < days; d++)
            printf("%-7d", *(matrix + s * days + d));
        printf("\n");
    }
}

void calculate_percentages(int *matrix, int students, int days)
{
    int s, d, present;
    double percentage;

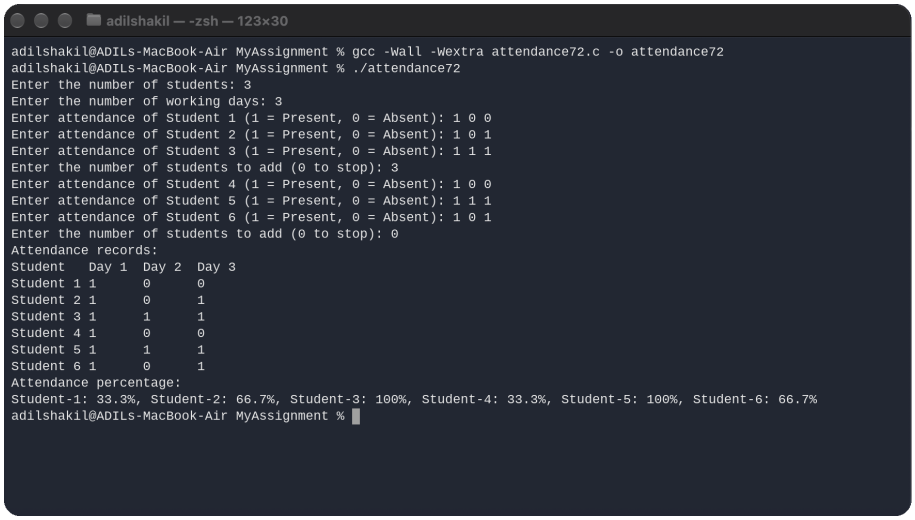
    printf("Attendance percentage:\n");
    for (s = 0; s < students; s++) {
        present = 0;
        for (d = 0; d < days; d++)
            present += *(matrix + s * days + d);
        percentage = present * 100.0 / days;
        printf("Student-%d: ", s + 1);
        if (percentage == (int)percentage)
            printf("%d%%", (int)percentage);
        else
            printf("%.1f%%", percentage);
        printf(s < students - 1 ? ", " : "\n");
    }
}

void free_matrix(int **matrix)
{
    free(*matrix);
    *matrix = NULL;
}
```

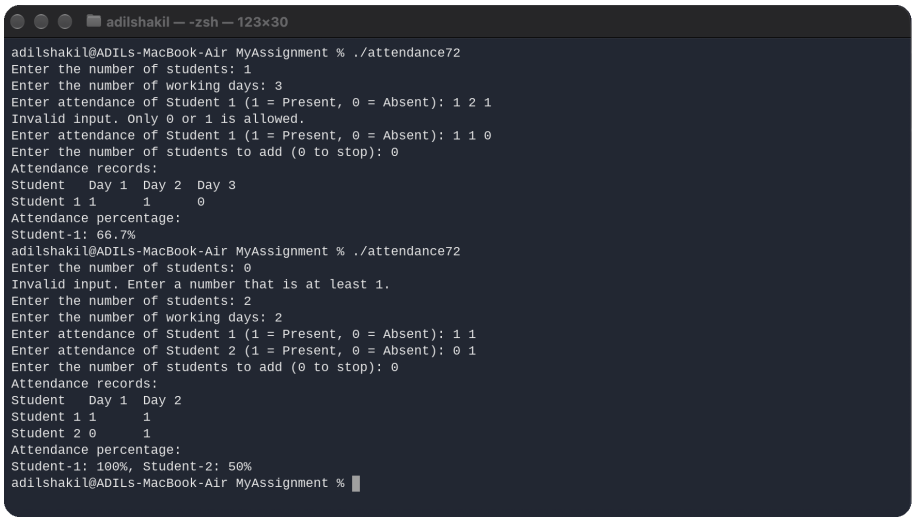
5. Compilation and Output

Compiled with `gcc -Wall -Wextra attendance72.c -o attendance72` so the small warnings would show up too. It built clean.

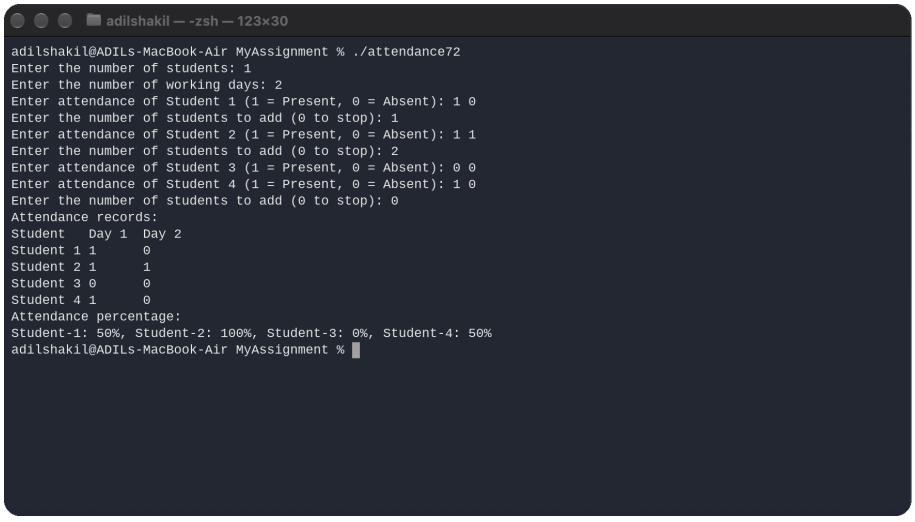
Test 1



Tests 2 and 3



Test 4



All four passed.